

栈

1. 栈的定义和基本操作

定义

栈是只能在一端（栈顶）进行插入和删除的线性表

先进后出

在没有限定操作序列的前提下，入栈序列和出栈序列会呈现一对多的关系

基本操作

- InitStack (&S) —— 初始化一个空栈
- StackEmpty(S) —— 判断栈是否空。若栈S为空，返回true；否则返回false
- Push(&S, x) —— 入栈操作。若栈未满，则x成为新的栈顶元素
- Pop(&S, &x) —— 出栈操作。若栈非空，则弹出栈顶元素，并通过x返回该值
- GetTop(S, &x) —— 读取栈项元素但不出栈。若栈非空，则通过x返回栈项元素
- Destroystack(&S) —— 销毁栈S，并释放其所占用的存储空间

2. 栈的顺序表示

顺序栈的定义

```
#define MaxSize 50 //定义顺序栈的最大容量为 50
typedef struct{
    ElemType data[MaxSize]; //用数组 data 存放栈中的元素
    int top; //栈顶指针，记录当前栈顶元素的位置
}SqStack; //顺序栈的结构类型为SqStack
```

顺序栈的指针设置



顺序栈的基本操作

- 初始化
 - void InitStack(SqStack &S){
 S.top = -1; //将栈顶指针设为 -1，表示当前栈为空
}
- 判栈空
 - bool StackEmpty(SqStack S){
 if(S.top == -1) //栈顶指针为-1，则栈空
 return true;
 else return false; //否则返回false，表示栈非空
}
- 入栈
 - bool Push(SqStack &S, ElemType x){
 if(S.top == MaxSize-1) //栈满，报错
 return false;
 S.data[++S.top] = x; //栈顶指针先加1，再把x存入新的栈顶位置
 return true; //入栈成功，返回true
}
- 出栈
 - bool Pop(SqStack &S, ElemType &x){
 if(S.top == -1) //栈空，报错
 return false;
 x = S.data[S.top--]; //先将栈顶元素读取到x中，再把栈顶指针减1
 return true; //出栈成功，返回true
}
- 读栈顶元素
 - bool GetTop(SqStack S, ElemType &x){
 if(S.top == -1) //栈空，报错
 return false;
 x = S.data[S.top]; //将栈顶元素读取到x中
 return true;
}

3. 栈的链式表示

链栈的定义

```
typedef struct StackNode{ //定义链栈中节点的结构体类型
    ElemType data; //data用来存放当前节点的数据元素
    struct StackNode *next; //指针next指向下一个节点
}StackNode, *LinkStack;
//StackNode是节点类型，LinkStack是指向节点的指针类型
```

链栈的基本操作

- 初始化
 - bool InitStack(LinkStack &S){
 S = NULL; //令栈顶指针为空，表示这是一个空栈
 return true; //初始化成功，返回true
}
- 入栈
 - bool Push(LinkStack &S, ElemType e){
 p = (StackNode*)malloc(sizeof(StackNode)); //为新节点申请一块存储空间
 p->data = e; //将入栈元素e存入新节点的数据域
 p->next = S; //让新节点的next指向原来的栈顶节点
 S = p; //更新栈顶指针s，使其指向新的栈顶节点
 return true; //入栈成功，返回true
}
- 出栈
 - bool Pop(LinkStack &S, ElemType &e){
 e = S->data; //先取出栈顶元素，保存到 e 中
 p = S; //用指针p暂存当前栈顶元素
 S = S->next; //栈顶指针后移，指向原栈顶节点的下一个节点
 free(p); //释放原栈顶节点所占的存储空间
 return true; //出栈成功，返回true
}
- 读栈顶元素
 - ElemType GetTop(LinkStack S){
 if(S != NULL) return S->data; //若栈非空，则返回栈顶元素
}

4. 卡特兰数

有n个元素入栈，那么有卡特兰数种不同的出栈方式

卡特兰数 $\frac{1}{n+1}C_{2n}^n$